

JTS PowerTool — Developer & Engineering Architecture

Technical Reference Guide: Stack Architecture, Security Scoping, Database Schemas, and API Specs.

Next.js 14 App Router

FastAPI Async Backend

PostgreSQL ORM

GitHub MCP

AWS Secrets Vault

1. Technology Stack & Architecture Blueprint

JTS PowerTool is engineered as a decoupled full-stack agent management controller bridging Slack event webhooks, Anthropic Claude API tool calling, GitHub Model Context Protocol (MCP) automation, PostgreSQL persistence, and AWS Secrets Manager.

LAYER	TECHNOLOGY	TECHNICAL FUNCTION
Frontend Console	Next.js 14 (App Router)	React 18 client/server components, TypeScript, Tailwind CSS, Lucide icons, SSE EventSource streaming interfaces.
Backend REST API	FastAPI + Uvicorn	Async Python web engine, HTTP middleware pipeline, session cookie authentication, SSE event streaming.
Database & Storage	PostgreSQL + SQLAlchemy	Relational storage for RBAC users, channel folders, pending write proposals, API call billing logs, and RAG contexts.
AI Engine	Anthropic Claude API	Claude 3.5 Sonnet / Haiku integration, structured tool calling schemas, system prompt RAG injection.
Automation Engine	GitHub MCP Server	Stdio-based Model Context Protocol server executing automated Git commits, PR creation, and repository modifications.
Secrets Engine	AWS Secrets Manager	Boto3 SDK integration storing encrypted API keys with zero raw secret keys saved in database tables.

2. Multi-Tenant RBAC & Security Isolation

Authentication is enforced globally via custom FastAPI middleware (`enforce_dashboard_authentication` in `app/main.py`).

Authentication & Session Pipeline

Session tokens are issued upon login and validated via HTTP-only cookies (`jts_session_token`) or `Authorization: Bearer <token>` headers.

ROLE LEVEL	SYSTEM ACCESS SCOPE	DATA ISOLATION RULE
<code>jts_admin</code>	Global Master Administrator	Bypasses tenant scoping. Full access to raw DB tables, SSE streaming logs, AWS Secrets Manager, and Claude Context snapshots.
<code>client_admin</code>	Client Organization Manager	Tenant-scoped via <code>client_folder_id</code> . Can manage team users, view client billing costs, and organize assigned channels. Cannot access system vault/logs/DB.
<code>client_standard</code>	Standard Client User	Restricted to <code>/my-tasks</code> portal and approving HITL write actions originating from their assigned channels.

3. System Workflows & Event Pipelines

3.1 Human-in-the-Loop (HITL) Proposal Execution Pipeline

To prevent unauthorized code mutations, write tools follow a human sign-off workflow:

1. User sends request via Slack or Web UI.
2. Claude Agent generates unified Git line diffs (`+` additions / `-` deletions).
3. Proposal is saved in `pending_approvals` with status `pending`.
4. Web UI displays unified code diff with **Approve & Commit** or **Reject** buttons.
5. Upon approval, FastAPI invokes GitHub MCP Server to commit and push changes.

3.2 Token Billing & Telemetry Engine

Every Claude API turn is intercepted by the telemetry logger:

- Extracts `input_tokens` and `output_tokens` from response headers.
- Calculates exact USD cost based on 1M token rate.
- Inserts log record into PostgreSQL table `api_usage_logs`.
- Pushes event to live SSE stream (`/logs/stream`).

4. Database Schema & Data Models

Defined using SQLAlchemy ORM in `app/db/models.py` :

TABLE NAME	PRIMARY COLUMNS	PURPOSE & DESCRIPTION
<code>dashboard_users</code>	<code>id</code> , <code>username</code> , <code>email</code> , <code>hashed_password</code> , <code>role</code> , <code>client_folder_id</code>	User accounts and multi-tenant role/folder bindings.
<code>channel_folders</code>	<code>id</code> , <code>name</code> , <code>description</code> , <code>created_at</code>	Permanent client/project organizational folders.
<code>channel_metadata</code>	<code>channel_id</code> , <code>channel_name</code> , <code>channel_type</code> , <code>folder_id</code>	Slack channel and DM folder mapping registry.
<code>pending_approvals</code>	<code>approval_id</code> , <code>tool_name</code> , <code>diff_preview</code> , <code>status</code> , <code>channel_id</code>	HITL write action proposals and execution results.
<code>api_usage_logs</code>	<code>id</code> , <code>workspace_id</code> , <code>channel_id</code> , <code>user_id</code> , <code>model</code> , <code>input_tokens</code> , <code>output_tokens</code> , <code>cost_usd</code>	Granular API token consumption & cost telemetry records.
<code>channel_secret_mappings</code>	<code>channel_id</code> , <code>provider</code> , <code>aws_secret_name</code> , <code>aws_secret_arn</code>	AWS Secrets Manager references. Zero raw secrets stored in DB.

5. Backend API Endpoints Directory

ROUTER / CATEGORY	ENDPOINT ROUTE	METHOD	ROLE ACCESS & FUNCTION
Auth Router	<code>/api/auth/login</code>	POST	Public. Validates credentials & issues session token cookie.
Users Router	<code>/api/users</code>	GET / POST	Admin. List or create dashboard users & bind client folders.
Folders Router	<code>/api/folders</code>	GET / POST	Admin. Manage client folders & sync Slack channels.
Approvals Router	<code>/api/approvals</code>	GET / POST	All Roles. Fetch approvals list & submit approve/reject action.

ROUTER / CATEGORY	ENDPOINT ROUTE	METHOD	ROLE ACCESS & FUNCTION
Usage & Billing	/api/usage/summary	GET	Admin. Get aggregated token usage and USD cost summaries.
Telemetry Stream	/logs/stream	GET (SSE)	JTS Admin. SSE stream emitting real-time log events.
Context Inspector	/api/context/snapshots	GET	JTS Admin. Inspect Claude prompt turns, system prompts & RAG context.
Database Explorer	/api/database/tables	GET / DELETE	JTS Admin. Direct table schema inspection, row delete & DB reset.

6. Developer Setup & Execution Blueprint

```
# 1. Environment Setup (.env)
DATABASE_URL=postgresql://user:password@localhost:5432/jts_powertool
ANTHROPIC_API_KEY=sk-ant-...
AWS_REGION=us-east-1
SLACK_BOT_TOKEN=xoxb-...

# 2. Start FastAPI Backend (Uvicorn)
python -m app.memory_manager
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# 3. Start Next.js Frontend
cd frontend
npm install
npm run dev # Running on http://localhost:3000
```